

Dictionaries and Sets

Introduction to dictionaries

dict()

- Function to create a dictionary
- also use {}
 - eg: my_dict = {}

```
zoo = dict(  
    pen_1 = 'monkeys',  
    pen_2 = 'zebras',  
    pen_3 = 'lions',  
)  
print(zoo['pen_2'])
```

```
##  
zebras
```

or

```
zoo = dict(  
    [  
        ['pen_1', 'monkeys'],  
        ['pen_2', 'zebras'],  
        ['pen_3', 'lions'],  
    ]  
)
```

```
zoo['pen_2']
```

```
##  
zebras
```

Immutable keys

- A dictionary's keys must be immutable
- Integers
- Floats
- Tuples
- Strings

Mutable data types cannot be used as keys

- Lists
- Sets
- Other dictionaries

Another important thing to note about dictionaries is that they're unordered. That means you can't access them by referencing a positional index.

you can check if a keyword exists in your dictionary simply by using the "IN" keyword.

```
print('pen_1' in zoo)
```

Dictionary methods

```
new_team = {}
for name, age, position in team:
    if position in new_team:
        new_team[position].append((name, age))
    else:
        new_team[position] = [(name, age)]

new_team

##output
```

```
{
  'center': [('Marta', 20), ('Sandra', 19)],
  'point guard': [('Ana', 22), ('Mari', 18)],
  'shooting guard': [('Gabi', 22), ('Esme', 18)],
  'power forward': [('Luz', 21), ('Lin', 18)],
  'small forward': [('Lorena', 19), ('Sol', 19)]
}
```

keys()

- A dictionary method to retrieve only the dictionary's keys

Reference guide: Dictionaries

By now you've encountered dictionaries and are discovering their power and utility as a data structure in Python. You've also learned that dictionaries provide a way to store and retrieve data using key-value pairs. Data professionals use dictionaries for many tasks, so it's important to be familiar with how they work. This reading is a reference guide about dictionaries. It's designed to help you in your Python learning journey.

Save this course item

You may want to save a copy of this guide for future reference. Use it as a resource for additional practice or in your future professional projects. To access a downloadable version of this course item, click the link below and select "Use Template."

[Reference guide: Dictionaries](#)

OR

If you don't have a Google account, download the item directly from the attachment below.

[Reference guide DictionariesDOCX File](#)

Create a dictionary

There are two main ways to create dictionaries in Python:

- Braces: {}

- The dict function: **dict()**

When instantiating a dictionary using braces, separate each element with a colon. For example, the following code creates a dictionary containing continents as keys and their smallest countries as values:

Note: The following code block is not interactive.

```
smallest_countries = {'Africa': 'Seychelles',
                      'Asia': 'Maldives',
                      'Europe': 'Vatican City',
                      'Oceania': 'Nauru',
                      'North America': 'St. Kitts and Nevis',
                      'South America': 'Suriname'
                      }
```

To create an empty dictionary, use empty braces or the **dict()** function:

Note: The following code block is not interactive.

```
empty_dict_1 = {}
empty_dict_2 = dict()
```

The **dict()** function uses a different syntax, where keys are entered as the function's keyword arguments and values are assigned with an equals operator:

Note: The following code block is not interactive.

```
smallest_countries = dict(africa='Seychelles',
                          asia='Maldives',
                          europe='Vatican City',
                          oceania='Nauru',
                          north_america='St. Kitts and Nevis',
                          south_america='Suriname'
                          )
```

Notice that, because the keywords cannot be entered as strings, they cannot contain whitespaces.

Some important notes about keys and values:

- **Dictionary keys:** Can be of any *immutable* data type, such as strings, numbers, or tuples
- **Dictionary values:** Can be of any data type—mutable or immutable—including other dictionaries or objects
- Each key can only correspond to a single value; so, for example, this will throw an error:

```
invalid_dict = {'numbers': 1, 2, 3}

##
Error on line 1:
    invalid_dict = {'numbers': 1, 2, 3}
                        ^
SyntaxError: invalid syntax
```

But if you enclose multiple values within another single data structure, you can create a valid dictionary. For example:

```
valid_dict = {'numbers': [1, 2, 3]}
print(valid_dict)

##
{'numbers': [1, 2, 3]}
```

Work with dictionaries

Access values

To access a specific value in a dictionary, you must refer to its key using brackets:

```
my_dict = {'nums': [1, 2, 3],
           'abc': ['a', 'b', 'c']}
print(my_dict['nums'])
```

```
##  
[1, 2, 3]
```

To access all values in a dictionary, use the **values()** method:

```
my_dict = {'nums': [1, 2, 3],  
          'abc': ['a', 'b', 'c']  
          }  
print(my_dict.values())  
  
##  
dict_values([[1, 2, 3], ['a', 'b', 'c']])
```

Assign new keys

Dictionaries are mutable data structures in Python. You can add to and modify existing dictionaries. To add a new key to a dictionary, use brackets:

```
my_dict = {'nums': [1, 2, 3],  
          'abc': ['a', 'b', 'c']  
          }  
  
# Add a new 'floats' key  
my_dict['floats'] = [1.0, 2.0, 3.0]  
print(my_dict)
```

Check if a key exists in a dictionary

To check if a key exists in a dictionary, use the **in** keyword:

```
smallest_countries = {'Africa': 'Seychelles',  
                     'Asia': 'Maldives',  
                     'Europe': 'Vatican City',  
                     'Oceania': 'Nauru',  
                     'North America': 'St. Kitts and Nevis',  
                     'South America': 'Suriname'  
                     }
```

```
print('Africa' in smallest_countries)
print('Asia' not in smallest_countries)

##
True
False
```

Delete a key-value pair

To delete a key-value pair from a dictionary, use the **del** keyword:

```
my_dict = {'nums': [1, 2, 3],
           'abc': ['a', 'b', 'c']}
del my_dict['abc']
print(my_dict)

##
{'nums': [1, 2, 3]}
```

Dictionary methods

Dictionaries are a core Python class. As you've learned, classes package data with tools to work with it. Methods are functions that belong to a class.

Dictionaries have a number of built-in methods that are very useful. Some of the most commonly used methods include:

items()

Return a view of the (key, value) pairs of the dictionary:

```
my_dict = {'nums': [1, 2, 3],
           'abc': ['a', 'b', 'c']}
print(my_dict.items())
```

```
##  
dict_items([('nums', [1, 2, 3]), ('abc', ['a', 'b', 'c'])])
```

keys()

Return a view of the dictionary's keys:

```
my_dict = {'nums': [1, 2, 3],  
          'abc': ['a', 'b', 'c']  
          }  
print(my_dict.keys())  
  
##  
dict_keys(['nums', 'abc'])
```

values()

Return a view of the dictionary's values:

```
my_dict = {'nums': [1, 2, 3],  
          'abc': ['a', 'b', 'c']  
          }  
print(my_dict.values())  
  
dict_values([[1, 2, 3], ['a', 'b', 'c']])  
  
##  
dict_values([[1, 2, 3], ['a', 'b', 'c']])
```

Note that the objects returned by these methods are view objects. They provide a dynamic view of the dictionary's entries, which means that, when the dictionary changes, the view reflects these changes. Dictionary views can be iterated over to yield their respective data. They also support membership tests.

Additional resources

- For more information about dictionaries, refer to the [Python dictionary documentation](#).
- For more dictionary methods, refer to the [Python mapping types documentation](#).
- For more information about view objects, refer to the [Python dictionary view objects documentation](#)

Introduction to sets

Set

- A data structure in Python that contains only unordered, non-interchangeable elements
- Instantiated with `set()` function or non-empty braces

`set()` function

- A function that takes an iterable as an argument and returns a new set object
- each element must be unique in set
- pass a string through the set function. It doesn't return the string, just the singular occurrence of the letters in the string

eg:

```
x = set('foo')
print(x)
```

```
##
{'o', 'f'}
```

```
#but
x = {'foo'}
print(x)
```

```
##
{'foo'}
```

To define an empty set, it's best to use the set function (set())

- keep in mind that because the elements inside a set are immutable, a set cannot be indexed or sliced.

intersection() - giống với phép giao

- A function that finds the elements that two sets have in common

eg:

```
set1 = {1,2,3,4,5,6}
set2 = {4,5,6,7,8,9}

print(set1.intersection(set2)) ## {4,5,6}
#or
print(set1 & set2) ##{4,5,6}
```

union()

- A function that finds all the elements from both sets

```
x1 = {'foo', 'bar', 'baz'}
x2 = {'baz', 'qux', 'quux'}
print(x1.union(x2))
print(x1 | x2)

##
{'quux', 'bar', 'foo', 'qux', 'baz'}
{'quux', 'bar', 'foo', 'qux', 'baz'}
```

difference()

- A function that finds the elements present in one set, but not the other

```
set1 = {1,2,3,4,5,6}
set2 = {4,5,6,7,8,9} → # in set1 but not in set2

print(set1.difference(set2))
```

```
print(set1 - set2)
```

```
##
```

```
{1, 2, 3}
```

```
{1, 2, 3}
```

symmetric_difference()

- A function that finds elements from both sets that are mutually not present in the other

```
set1 = {1, 2, 3, 4, 5, 6}
```

```
set2 = {4, 5, 6, 7, 8, 9}
```

```
print(set2.symmetric_difference(set1))
```

```
print(set2 ^ set1)
```

```
##
```

```
{1, 2, 3, 7, 8, 9}
```

```
{1, 2, 3, 7, 8, 9}
```

Reference guide: Sets

Data professionals depend on sets for separating data and identifying its unique elements. As you have been discovering, set objects are similar to lists and dictionaries, yet they do not have key-value pairs or positional index[i] capability. Additionally, sets contain unique values but have no item order or index behavior. Data professionals compare sets to understand the range of data they contain, where they intersect, and what items are present in either set but not both. Sets are also helpful when cleaning data for analysis. This reading is a reference guide for sets to help you as you continue learning Python.

Save this course item

You may want to save a copy of this guide for future reference. Use it as a resource for additional practice or in your future professional projects. To

access a downloadable version of this course item, click the link below and select "Use Template."

Reference guide: Sets

OR

If you don't have a Google account, download the item directly from the attachment below.

Reference guide_ SetsDOCX File

Sets review

A set is a collection of unique data elements, without duplicates. In Python, it is an object class—in fact, two different classes—which you'll learn about in this reading. However, sets are not unique to Python or even to computer programming; they are an important concept in general mathematics. Sets provide a simple means to identify unique data elements.

Create a set

Create a set using braces:

```
my_set = {5, 10, 10, 20}
```

Note that an empty set cannot be created with braces, as this will be interpreted as an empty dictionary.

There are two functions for creating sets in Python: **set()** and **frozenset()**. Use these on any iterable object. Or use these functions to create empty sets.

set()

- This is a mutable data type.
- Because it's mutable, this class comes with additional methods to add and remove data from the set.
- It can be applied to any iterable object and will remove duplicate elements from it.
- It is **unordered** and **non-indexable**.
- Elements in a set must be hashable; generally, this means they must be immutable. (Refer to the additional resources for more information on hashing.)

In the examples that follow, four sets are instantiated using a variety of data types:

```
example_a = [1, 2, 2.0, '2']
set(example_a)

##
{1, 2, '2'}
```

Notice that, in the preceding example, 2 and 2.0 are evaluated as equivalent, even though one is an integer and the other is a float.

```
example_b = ('apple', (1, 2, 2, 2, 3), 2)
set(example_b)

##
{2, 'apple', (1, 2, 2, 2, 3)}
```

In the preceding example, **(1, 2, 2, 2, 3)** is a tuple, which is hashable ($\approx$ immutable) and thus treated as a distinct single element in the resulting set.

```
example_c = [1.5, {'a', 'b', 'c'}, 1.5]
set(example_c)

##
Error on line 2:
  set(example_c)
TypeError: unhashable type: 'set'
```

The preceding example throws an error because each element of a set must be hashable ($\approx$ immutable), but **{'a', 'b', 'c'}** is a set, which is a mutable (unhashable) object.

The following example demonstrates the **add()** method, which is one of the special methods available to sets but not to frozensets.

```
example_d = {'mother', 'hamster', 'father'}
example_d.add('elderberries')
example_d
```

```
##  
{'father', 'hamster', 'mother', 'elderberries'}
```

An element was added to the **example_d** set, thus modifying it. This is an example of the mutability of the **set** class.

frozenset()

Frozensets are another type of set in Python. They are their own class, and [they are very similar to sets, except they are immutable](#).

- This is an immutable data type.
- It can be applied to any iterable object and will remove duplicate elements from it.
- Because they're immutable, [frozensets can be used as dictionary keys and as elements in other sets](#).

In this example, a frozenset is used within a set.

```
example_e = [1.5, frozenset(['a', 'b', 'c']), 1.5]  
set(example_e)  
  
##  
{1.5, frozenset({'c', 'a', 'b'})}
```

Unlike **example_c** previously, this set does not throw an error. This is because it contains a frozenset, which is an immutable type and can therefore be used in sets.

Set methods

Sets are useful to determine which values are contained in a data structure and to eliminate duplicate values. There are numerous set methods—such as intersection, union, difference, and symmetric difference—that add functionality and power to working with sets.

union()

- Return a new set with elements from the set and all others.

- The operator for this function is the pipe (`|`).

```
set_1 = {'a', 'b', 'c'}
set_2 = {'b', 'c', 'd'}

print(set_1.union(set_2))
print(set_1 | set_2)

##
{'d', 'c', 'a', 'b'}
{'d', 'c', 'a', 'b'}
```

intersection()

- Return a new set with elements common to the set and all others.
- The operator for this function is the ampersand (`&`).

```
set_1 = {'a', 'b', 'c'}
set_2 = {'b', 'c', 'd'}

print(set_1.intersection(set_2))
print(set_1 & set_2)

##

{'b', 'c'}
{'b', 'c'}
```

difference()

- Return a new set with elements in the set that are not in the others.
- The operator for this function is the subtraction operator (`-`).

```
set_1 = {'a', 'b', 'c'}
set_2 = {'b', 'c', 'd'}

print(set_1.difference(set_2))
print(set_1 - set_2)
```

symmetric_difference()

- Return a new set with elements in either the set or other, but not both.
- The operator for this function is the caret (^).

```
set_1 = {'a', 'b', 'c'}
set_2 = {'b', 'c', 'd'}

print(set_1.symmetric_difference(set_2))
print(set_1 ^ set_2)

##
{'d', 'a'}
{'d', 'a'}
```

Additional resources

- Refer to the Python documentation for more information about [sets and frozensets](#), including a complete list of available class methods.
- For methods unique to sets (and unavailable to frozensets), refer to this [Python set methods documentation](#).
- For more examples of sets, refer to the [Python tutorial on sets](#).
- For more information on hash tables, what makes something hashable, and hashing as a concept, refer to this [resource from Runestone Academy](#). For an interesting story about the birth of the original hashing algorithms, check out this [IEEE Spectrum article](#)